Orientações ao Roger – Software de aquisição e estabilização da Maca Inteligente

Roger, com base nas últimas calibrações realizadas por Tiago e Fabrício, precisamos ajustar o software para separar três problemas diferentes: aquisição do sinal, estabilização da leitura e conversão da leitura em massa. Neste momento, a prioridade é tornar os dois primeiros robustos. O modelo definitivo raw → massa será fechado depois da consolidação dos experimentos.

1. Não tratar o valor RAW como tensão, resistência ou força

O dado recebido da manta deve ser armazenado simplesmente como:

$$s_{ij}(t) = \text{resposta digital RAW da célula } (i, j).$$

Portanto, por enquanto, não implementar conversões do tipo:

$$RAW \rightarrow V \rightarrow R \rightarrow G \rightarrow F$$

porque ainda não temos documentação suficiente da eletrônica interna para justificar fisicamente essa cadeia.

No código e banco de dados, utilizar nomes como:

raw_value, raw_mean, raw_std, baseline_raw

e evitar voltage, resistance ou force, salvo se a grandeza estiver realmente disponível.

---

2. Implementar baseline espacial da manta vazia

Em vez de uma única tara global, manter uma matriz:

$$B_{ij}$$

correspondente ao baseline de cada elemento.

Durante a tara:

$$B_{ij} = \frac{1}{N} \sum_{k=1}^{N} s_{ij}\,(k).$$

A resposta corrigida será:

$$x_{ij}(t) = s_{ij}(t) - B_{ij}.$$

Se:

$$x_{ij} < 0,$$

pode-se inicialmente utilizar:

$$x_{ij} = 0.$$

Mas é importante guardar também o RAW original, para análise posterior.

A tara deve ser executada somente quando o sistema tiver certeza de que a maca está vazia.

3. Manter a tara de 5 segundos, mas torná-la parametrizável

O mecanismo atual de Roger, que utiliza aproximadamente 5 s de aquisição para tara, pode permanecer.

Mas transformar em parâmetro:

T_TARE = 5 s

para posteriormente compararmos:

$$5,\ 10,\ 15s.$$

Também guardar:

timestamp_tare

tare_id

baseline_mean

baseline_std

model_version

configuration_id.

4. Criar uma máquina de estados

Esta é provavelmente a modificação mais importante.

Sugiro:

SEM_CARGA

↓

CARGA_DETECTADA

↓

ESTABILIZANDO

↓

ESTÁVEL

↓

MEDIÇÃO_VÁLIDA.

Se ocorrer movimento significativo:

ESTÁVEL → MOVIMENTO → ESTABILIZANDO.

O sistema não deve publicar novo peso clínico durante MOVIMENTO ou ESTABILIZANDO.

5. Implementar detecção matemática de estabilidade

Para cada janela temporal, calcular pelo menos:

$$\mu(t) = \text{média da janela}$$

e

$$\sigma(t) = \text{desvio-padrão}.$$

Além disso:

$$D(t) = | \mu(t) - \mu(t - \Delta t) |.$$

A condição geral de estabilidade será:

$$\sigma(t) < \sigma_{\max}$$

e

$$D(t) < D_{\max}$$

durante:

$$T_{\min}.$$

Por enquanto, $\sigma_{\max}$, $D_{\max}$e $T_{\min}$devem ser configuráveis, e não constantes definitivas no código.

Exemplo:

STABILITY_WINDOW = configurável

STD_THRESHOLD = configurável

DELTA_THRESHOLD = configurável

MIN_STABLE_TIME = configurável

Isso permitirá aos experimentos determinarem os valores corretos.

6. Para a manta inteira, medir movimento espacial

Não basta analisar apenas a soma total.

Calcular:

$$D_M(t) = \sum_{i,j} | x_{ij}(t) - x_{ij}(t - \Delta t) |.$$

Esse indicador permitirá identificar redistribuição de pressão causada pelo movimento do paciente.

Quando:

$$D_M > D_{\text{mov}},$$

o sistema entra em:

PACIENTE EM MOVIMENTO / AGUARDANDO ESTABILIZAÇÃO

e mantém o último peso validado.

7. Separar “estável” de “correto”

Isto é fundamental diante das últimas calibrações.

Uma leitura pode apresentar:

$$\sigma \approx 0$$

e mesmo assim não corresponder exatamente à massa real.

Portanto, o software deve possuir dois conceitos independentes:

Sinal estável

e

Massa validada.

Nunca usar:

$$\text{estabilidade} \equiv \text{exatidão}.$$

8. Registrar o tempo de estabilização

A partir da aplicação da carga:

$$t_0$$

até o instante em que o critério de estabilidade é satisfeito:

$$t_s,$$

registrar:

$$T_s = t_s - t_0.$$

Campo:

stabilization_time_s

Isso permitirá responder experimentalmente se devemos esperar:

$$10,\ 20,\ 30,\ 45 \text{ ou } 60s.$$

A proposta do Prof. Walter é justamente explorar o regime estabilizado e, se necessário, trabalhar com ramos diferentes de carga e descarga.

9. Registrar subida e descida

No módulo experimental, incluir:

loading_direction = UP

loading_direction = DOWN

e, quando não for possível determinar:

loading_direction = UNKNOWN.

Uma possibilidade futura é estimar a direção por:

$$\frac{\Delta s}{\Delta t},$$

como também aparece na proposta do Prof. Walter.

Mas não utilizar ainda isso automaticamente para corrigir peso clínico. Primeiro vamos verificar se, após estabilização, ainda será necessário manter duas curvas.

10. Criar indicador de qualidade da medição

Eu acrescentaria ao software um conceito novo:

$$Q_{\text{medição}}.$$

Não precisa ser imediatamente um número de 0–100. Inicialmente pode ser um conjunto de flags:

TARE_OK

BASELINE_OK

SIGNAL_STABLE

MOVEMENT_FALSE

SATURATION_FALSE

MISSING_CELLS_FALSE

OUT_OF_CALIBRATION_FALSE

MODEL_VALID

Somente quando as condições necessárias forem satisfeitas:

WEIGHT_VALID = TRUE

Caso contrário, em vez de apresentar um número aparentemente preciso:

PESO NÃO VALIDADO — AGUARDANDO ESTABILIZAÇÃO

Isso é muito melhor do que apresentar, por exemplo, 70,34 kg quando o algoritmo sabe que a condição da leitura é inadequada.

---

11. Preparar desde agora a detecção de região de baixa sensibilidade

Os experimentos mostram que a resposta digital perde progressivamente sensibilidade com o aumento da carga local.

Roger deve deixar a arquitetura pronta para cada célula possuir uma região de calibração:

cell_id

raw_min_valid

raw_max_valid

mass_min_valid

mass_max_valid

calibration_version

Se determinada célula entrar numa região na qual a calibração futura demonstrar baixa capacidade de discriminação:

LOW_SENSITIVITY = TRUE.

Não devemos definir ainda numericamente onde começa essa região.

12. Não interpolar célula saturada pelos vizinhos neste momento

Aqui eu faria uma pequena diferença em relação ao documento do Prof. Walter.

Ele sugere, no esqueleto computacional, interpolar células saturadas pelos vizinhos.

Eu não faria isso automaticamente agora.

Primeiro:

SATURATION_FLAG = TRUE

e registrar a ocorrência.

Interpolar espacialmente pode ser útil para reconstruir um mapa de pressão, mas para estimar massa total, inventar uma contribuição de carga com base nos vizinhos pode introduzir erro sistemático.

Vamos estudar isso separadamente.

---

13. Preparar a arquitetura para diferentes modelos de calibração

Roger não deve codificar Newton, potência ou ML como se um deles fosse definitivamente o modelo.

Criar uma interface conceitual:

$$\boxed{\hat{m}_{ij} = f(s_{ij}; \theta_{ij})}$$

de modo que f() possa posteriormente ser:

PCHIP

NEWTON

POWER

EXPONENTIAL

ISOTONIC

REGRESSION

ML

sem modificar todo o restante do sistema.

Isso é muito importante.

A aquisição não pode depender do modelo de calibração.

14. Manter os dados RAW

Isso é obrigatório para nossa pesquisa.

Nunca salvar apenas:

$$\hat{m}.$$

Guardar:

$$\boxed{RAW \rightarrow FILTRADO \rightarrow CORRIGIDO \rightarrow FEATURES \rightarrow ESTIMATIVA}$$

Assim, se mudarmos o modelo posteriormente, poderemos reprocessar os experimentos antigos sem precisar repetir tudo.

15. Estrutura mínima do registro experimental

Eu pediria ao Roger que cada registro possua pelo menos:

timestamp

session_id

cycle_id

configuration_id

cell_id

row

column

raw_value

baseline_raw

corrected_raw

raw_mean

raw_std

loading_direction

known_mass_g

stabilization_time_s

stability_flag

movement_flag

saturation_flag

calibration_version

model_version

estimated_mass

prediction_valid

Para o sistema completo, acrescentar:

active_cells

total_raw

center_x

center_y

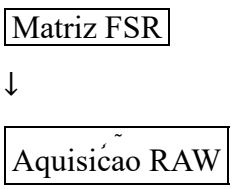
spatial_dispersion

movement_index

---

16. O pipeline que eu recomendaria ao Roger

A arquitetura deveria ficar assim:



↓

Baseline/Tara

↓

Filtragem temporal

↓

Detecc̃ao de movimento

↓

Teste de estabilidade

↓

Extrac̃ao de caracterıśticas

↓

Modelo de calibrac̃ao

↓

Validac̃ao da medic̃ao

↓

API

↓

Dashboard SAMEL

Isso separa completamente a física da aquisição do algoritmo matemático.

17. E eu pediria uma mudança na API

Além de retornar:

weight_kg

ela deveria retornar algo como:

weight_kg

weight_status

stable

stabilization_time_s

movement

measurement_quality

calibration_version

timestamp

Exemplo conceitual:

weight_kg: 70.42

weight_status: VALID

stable: true

stabilization_time_s: 18.4

movement: false

calibration_version: MC-2026-002

E durante movimentação:

weight_status: STABILIZING

stable: false

movement: true

mantendo separadamente o último peso válido.

18. O que Roger NÃO deve fazer ainda

Neste momento eu seria explícito com ele: não recalibrar automaticamente o ML com todos os dados disponíveis; não escolher o modelo apenas pelo maior R^2; não converter RAW em força física sem conhecermos a cadeia eletrônica; não misturar subida e descida indiscriminadamente; não considerar estabilidade como prova de exatidão; não descartar os dados brutos depois de calcular a massa.

A análise do Prof. Walter também propõe calibração individual, segmentação dos elementos ativos e posterior soma das contribuições locais. Essa arquitetura pode ser

preparada por Roger agora, mas a função matemática definitiva de conversão deve permanecer substituível.

A mensagem central para Roger

“Roger, não precisamos ainda que o software adivinhe melhor o peso; precisamos primeiro que ele saiba reconhecer quando possui uma medição confiável para tentar estimar o peso.”

Quando conseguirmos:

$$\boxed{\text{aquisição confiável} + \text{baseline confiável} + \text{estabilidade confiável}}$$

aí aplicaremos os dados de Tiago e Fabrício para resolver:

$$\boxed{\text{RAW estável} \rightarrow \text{massa local}}$$

e posteriormente:

$$\boxed{\{\hat{m}_{ij}\} \rightarrow \text{massa total do paciente}}.$$

Essa ordem evita que o modelo matemático seja obrigado a compensar problemas que, na realidade, pertencem à mecânica, à histerese, à tara ou à aquisição.